

Expectation in custom layer: `ValueError: Shape must be rank 2 but is rank 3`

<https://github.com/tensorflow/quantum/issues/695>

ROW 134

Expectation in custom layer: ValueError: Shape must be rank 2 but is rank 1 #695

[New issue](#)[Closed](#)

dbcq opened on Jun 30, 2022 · edited by dbcq

Edits ...

Hi, I'm using Expectation within a custom circuit layer, and I'm getting some shape errors:

```
ValueError: Shape must be rank 2 but is rank 1 for '{{node circuit_7/expectation_7/TfqSimulateExpectation}} = TfqSimulateExpectation[](circuit_7/expectation_7/Tile, circuit_7/expectation_7/Const, circuit_7/concat, circuit_7/expectation_7/234)' with input shapes: [?], [8], [?,8], [4].
```

I expect this has quite a straightforward solution but I can't work out what it actually wants to have rank 2 rather than rank 1!

The below should be a complete toy example:

```
import tensorflow as tf
import tensorflow_quantum as tfq
import cirq
import sympy
import numpy as np

data = [[1,1,1,1]]*4
data = np.array(data, dtype=np.float32)
labels = [1, 2, 3, 4]
labels = np.array(labels, dtype=int)
expectation = tfq.layers.Expectation()

class Circuit(tf.keras.layers.Layer):
    def __init__(self):
        super().__init__()
        self.encoding_syms = sympy.symbols('sym0:4')
        self.pqc_syms = sympy.symbols('syms0:4')
        variable_initializer = tf.random_uniform_initializer(0, 2*np.pi)
        self.pqc_params = tf.Variable(initial_value = variable_initializer(shape=(4,), dtype=tf.float32),
                                     name="pqc", trainable=True)

        self.qubits = [cirq.GridQubit(0, i) for i in range(4)]
        self.readout = [cirq.Z(self.qubits[i]) for i in range(4)]
        self.readout = tfq.convert_to_tensor(self.readout)

        self.encoding_circuit = self.get_encoding_circuit(self.qubits, self.encoding_syms)
        self.pqc_circuit = self.get_classifier_circuit(self.qubits, self.pqc_syms)
        self.full_circuit = self.encoding_circuit + self.pqc_circuit

    def get_encoding_circuit(self, qubits, symbols):
        circuit = cirq.Circuit()
        for i in range(4):
            circuit += cirq.rx(symbols[i]).on(qubits[i])
        return circuit

    def get_classifier_circuit(self, qubits, symbols):
        circuit = cirq.Circuit()
        for i in range(4):
            circuit += cirq.CZ(qubits[i], qubits[(i+1)%3])
        for i in range(4):
            circuit += cirq.ry(symbols[i]).on(qubits[i])
        return circuit

    def call(self, inputs):
        num_batch = tf.gather(tf.shape(inputs), 0)

        pqc_params = tf.tile([self.pqc_params], [num_batch, 1])
        all_params = tf.concat([inputs, pqc_params], 1)

        out = expectation(self.full_circuit,
                          symbol_names = self.encoding_syms + self.pqc_syms,
                          symbol_values = all_params,
                          operators=self.readout)

        return tf.sigmoid(out)

inputs = tf.keras.layers.Input(shape=(4,), dtype=tf.float32)
outs = Circuit()(inputs)
outs = tf.keras.layers.Dense(1)(outs)
```

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#) [Subscribe](#)

You're not receiving notifications from this thread.

Participants



```

circuit = Cirq.Circuit()
for i in range(4):
    circuit += cirq.CZ(qubits[i], qubits[(i+1)%3])
for i in range(4):
    circuit += cirq.ry(symbols[i]).on(qubits[i])
return circuit

def call(self, inputs):
    num_batch = tf.gather(tf.shape(inputs), 0)

    pqc_params = tf.tile([self.pqc_params], [num_batch, 1])
    all_params = tf.concat([inputs, pqc_params], 1)

    out = expectation(self.full_circuit,
                      symbol_names = self.encoding_syms + self.pqc_syms,
                      symbol_values = all_params,
                      operators=self.readout)

    return tf.sigmoid(out)

inputs = tf.keras.layers.Input(shape=(4,), dtype=tf.float32)
outs = Circuit()(inputs)
outs = tf.keras.layers.Dense(1)(outs)

model = tf.keras.models.Model(inputs=inputs, outputs=outs)

model.compile(
    optimizer='adam',
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=['accuracy'],
    run_eagerly=False
)

model.fit(data, labels, epochs=2, batch_size=2)`

```

Thanks in advance!



dbcq on Jun 30, 2022

Author ...

I've found the issue, sorry! It seems that converting the readout operators to a tensor was the issue.



dbcq closed this as **completed** on Jun 30, 2022